

**VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING**



**REPORT
CAPSTONE PROJECT
SEMESTER 2 ACADEMIC YEAR 2026-2027**

<TITLE>

MAJOR: COMPUTER SCIENCE
COUNCIL: <NAME>
SUPERVISOR(s): PhD. DIEP THANH DANG
REVIEWER:
——o0o——
STUDENT 1: NGO QUANG HIEU - 2311015

HO CHI MINH CITY, December 2027

Declaration of Authenticity

We declare that we solely conducted this specialized project, under the supervision of PhD. Diep Thanh Dang at the Faculty of Computer Science and Engineering, Vietnam National University - Ho Chi Minh City University of Technology.

We have taken care to properly acknowledge and document all external sources and references used in the project.

If there is any instance of plagiarism, we are ready to accept the consequences. Ho Chi Minh City University of Technology - Vietnam National University HCMC will not be held responsible for any copyright violations that may have occurred during my research.

Ho Chi Minh City, December 2027

Authors,

Ngo Quang Hieu

Acknowledgement

We would like to express my appreciation to Dr. Diep Thanh Dang for his invaluable guidance. Our research has greatly benefited from his deep knowledge, perceptive criticism, and constant support.

Besides, we are grateful for the help from ... for valuable comments for research aspect and our proposed pipeline, including pointing out strengths as well as weaknesses of previous works on multimodal models so that we can elaborate on our ideas more effectively.

In addition, we would like to extend our gratitude to my family. Their unwavering faith in our abilities and constant encouragement have been my pillars of strength. Their belief in my potential has been a constant source of motivation and resilience. We are forever grateful for their love and support.

Abstract

Contents

1	Introduction	1
1.1	Motivation	1
1.2	Goals	1
1.3	Scope	1
1.4	Thesis Structure	1
2	Related Works	3
2.1	Alternative Technologies	3
2.1.1	Stream Processing: Apache Flink vs. Apache Spark Structured Streaming	3
2.1.2	Storage & Warehousing: Data Lakehouse (Hudi) vs. Cloud Data Warehouse (Snowflake)	4
2.1.3	Processing Paradigm: Streaming vs. Plain Batch ETL	4
3	Theoretical Background	5
3.1	Core Theory and Concepts	5
3.1.1	Event Streaming and Publish-Subscribe Messaging	5
3.1.2	Spark Structured Streaming and Micro-batching	6
3.1.3	Table Formats: Apache Hudi (MERGE_ON_READ vs. COPY_ON_WRITE)	6
3.1.4	Star-Schema Dimensional Modeling	7
4	Problem Statement and Challenges	8
4.1	Dataset	8
4.2	Technical Challenges	8
4.2.1	Streaming Volume and Velocity	8

4.2.2	Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster	9
4.2.3	Computing and Partitioning on a Small, Shared Spark Cluster	9
5	Proposed Solution	10
5.1	System Architecture	10
5.2	Implementation Workflow	12
5.2.1	Star Schema and Wide Table	13
5.2.2	Streaming Job: Kafka to Hudi Bronze	14
5.3	Cluster and Environment Setup	15
6	Evaluation and Experiments	16
6.1	Test Environment	16
6.2	Test Cases	16
6.3	Monitoring as Qualitative Evidence	17
6.4	Scalability	17
7	Conclusion	19
7.1	Results Achieved	19
7.2	Limitations	19
7.3	Future Directions	20
	References	20

List of Tables

5.1	Technology stack per pipeline layer.	11
6.1	Scalability results (to be measured).	18

List of Figures

1	Streamify architecture overview: Eventsim, Kafka, Spark Structured Streaming, Hudi, dbt, Superset, and monitoring.	10
2	Star schema: fact_streams and its five dimension tables.	13
3	dbt DAG showing the dependency between bronze sources, the star schema, and wide_streams.	14
4	Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.	17
5	Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.	17

Chapter 1

Introduction

In chapter 1, the overview, objectives, and goals of the research project are illustrated. The outline of the report is also presented.

1.1 Motivation

1.2 Goals

1.3 Scope

1.4 Thesis Structure

The remainder of this report is organized as follows:

- **Chapter 2, Related Works**, reviews alternative technologies considered for each stage of the pipeline and the reasons for the choices made in this project.
- **Chapter 3, Theoretical Background**, covers the core concepts behind the pipeline: event streaming and pub-sub messaging, Spark Structured Streaming’s micro-batch model, Hudi’s MERGE_ON_READ and COPY_ON_WRITE table formats, and star-schema dimensional modeling.
- **Chapter 4, Problem Statement and Challenges**, describes the dataset used and the technical challenges of building a fault-tolerant streaming pipeline on a small,

shared compute cluster.

- **Chapter 5, Proposed Solution**, presents the system architecture, the implementation workflow, and the cluster/environment setup.
- **Chapter 6, Evaluation and Experiments**, describes the test environment, test cases, and monitoring evidence gathered.
- **Chapter 7, Conclusion**, summarizes the results achieved, current limitations, and directions for future work.

Chapter 2

Related Works

2.1 Alternative Technologies

Several alternative technologies were considered for each stage of the pipeline before settling on the stack described in Chapter 5.

2.1.1 Stream Processing: Apache Flink vs. Apache Spark Structured Streaming

Apache Flink [1] is a distributed stream-processing engine built around native, event-at-a-time streaming with low end-to-end latency. An earlier version of this project used a Flink SQL job for the Kafka-to-lakehouse ingestion path. Apache Spark Structured Streaming [5], in contrast, models streaming as a sequence of small batch jobs (micro-batching) on top of the same engine used for batch and SQL workloads.

The project replaced the Flink job with a Spark Structured Streaming job (Section 3.1) for two practical reasons:

- **Unified engine.** dbt already needed a Spark cluster (via the Spark Thrift Server) for the batch transformation layer. Running the streaming ingestion job on the same Spark Standalone cluster avoids operating two separate distributed engines (Flink and Spark) on a cluster with only 4 cores in total.
- **Native Hudi integration.** Apache Hudi ships first-class Spark data source sup-

port, which simplifies writing `MERGE_ON_READ` bronze tables directly from the streaming job.

The trade-off is latency: Flink’s native streaming model can achieve lower per-event latency than Spark’s micro-batch model. For this project’s throughput and cluster size, the micro-batch trade-off was considered acceptable.

2.1.2 Storage & Warehousing: Data Lakehouse (Hudi) vs. Cloud Data Warehouse (Snowflake)

Cloud data warehouses such as Snowflake [7] offer a fully managed SQL warehouse with separate storage and compute, but couple the project to a proprietary, hosted platform and its billing model. This project instead adopted the lakehouse pattern: raw and modelled data are stored as Apache Hudi tables on object storage (MinIO locally, Amazon S3 in `aws` mode), and Spark/dbt query that storage directly. This keeps the storage layer open-format and self-describing (each Hudi table carries its own `.hoodie/` metadata directory, so no separate catalog service is required) and keeps the whole stack deployable with Docker Compose on a single machine or a small cluster, which suited the project’s development and demonstration constraints better than a managed cloud warehouse.

2.1.3 Processing Paradigm: Streaming vs. Plain Batch ETL

A simpler alternative would have been a plain batch ETL job: periodically read newly arrived Eventsim output files and load them into the warehouse. This project instead used an always-on streaming pipeline (Kafka plus Spark Structured Streaming) so that events are captured and land in the bronze layer continuously, closer to real time, and so that the project could demonstrate streaming-specific concerns – partitioning, replication, checkpointed exactly-once processing, and consumer lag monitoring – that a batch-only design would not exercise.

Chapter 3

Theoretical Background

3.1 Core Theory and Concepts

This section summarizes the underlying concepts of the technologies used to build the pipeline in Chapter 5.

3.1.1 Event Streaming and Publish-Subscribe Messaging

Event streaming treats data as an unbounded sequence of immutable events (e.g. “user played a song”) rather than as static rows in a table. Producers append events to a log; one or more consumers read that log independently and at their own pace. Apache Kafka [3] implements this as a publish-subscribe (pub-sub) system: producers publish events to named *topics*, and consumers subscribe to the topics they need, decoupling producers from consumers in time and in number.

Partitioning and Replication

A Kafka topic is split into *partitions*, each an ordered, append-only log. Partitioning is Kafka’s unit of parallelism: different partitions of the same topic can be produced to and consumed from concurrently, and ordering is only guaranteed within a single partition. Each partition can additionally be *replicated* across multiple brokers (replication factor); one replica is the leader that serves reads and writes, and the others are followers that copy the leader’s log. If a broker holding the leader replica fails, a follower on a surviving

broker can take over, so the topic keeps serving as long as at least one replica of each partition survives.

3.1.2 Spark Structured Streaming and Micro-batching

Apache Spark Structured Streaming [5] runs a streaming query as a series of small, discrete batch jobs (*micro-batches*) on Spark's existing batch execution engine, rather than processing each event individually. On a fixed trigger interval, the engine reads whatever new data has arrived since the last batch (tracked via checkpointed offsets), applies the same DataFrame/SQL transformations used for batch queries, and writes the result. This design reuses Spark's mature batch engine (query planning, fault tolerance, exactly-once output via checkpointing) at the cost of end-to-end latency being bounded by the micro-batch interval rather than being event-at-a-time.

3.1.3 Table Formats: Apache Hudi (MERGE_ON_READ vs. COPY_ON_WRITE)

Apache Hudi [2] is a transactional table format for data lakes: it adds row-level insert/update/delete semantics, ACID commits, and time-travel to plain files on object storage (S3, MinIO, HDFS, ...). Hudi supports two table types that trade off write cost against read cost:

- **COPY_ON_WRITE (COW).** Every write rewrites the affected data files as new versioned Parquet files. Reads are fast (plain Parquet, no merge step), but writes are more expensive because entire files are rewritten on update.
- **MERGE_ON_READ (MOR).** Writes are appended to a compact row-based delta log next to the existing base Parquet files; the two are merged only at read time (or during a background compaction job). Writes are cheap and low-latency, at the cost of extra work on read (or a periodic compaction step) to materialize the merged view.

MOR suits high-frequency streaming appends, where write latency matters most; COW suits tables that are rebuilt in full on a schedule and then read many times, where read performance matters most.

3.1.4 Star-Schema Dimensional Modeling

Dimensional modeling organizes an analytical warehouse around a central *fact table* of measurable events (here, one row per stream/play event) surrounded by *dimension tables* of descriptive attributes (who, what, where, when) that the fact table references by key. This “star” shape – one fact table joined to several dimension tables – normalizes descriptive attributes into reusable dimensions while keeping the fact table narrow, and is the standard layout for BI and reporting workloads because it keeps common queries (aggregate the fact table, filter/group by a dimension) to a small, predictable number of joins.

Chapter 4

Problem Statement and Challenges

4.1 Dataset

The pipeline is driven by Eventsim, a synthetic event generator that simulates page-request and listening activity on a fake music-streaming website (the Docker image used is viirya's fork of the original, unmaintained project). Eventsim draws its song catalog from the Million Song Dataset; this project uses the official 10,000-song subset. The generated events (`listen_events`, `page_view_events`, `auth_events`) look like real user-activity logs but are entirely synthetic, which makes them safe to publish and reproduce without any user-privacy concern.

4.2 Technical Challenges

Building a streaming lakehouse pipeline on top of this dataset raised the following challenges.

4.2.1 Streaming Volume and Velocity

Eventsim continuously emits events across three topics, so the pipeline must ingest, decode, and land data in the lakehouse without falling behind the producer. This requires a message queue that can buffer and parallelize consumption (Kafka, partitioned per topic) and a streaming engine that can keep its micro-batch processing time below the

arrival rate.

4.2.2 Exactly-once, Fault-tolerant Processing on a 2-node Kafka Cluster

Kafka runs as a 2-node KRaft cluster (no ZooKeeper), with each topic replicated across both brokers (replication factor 2, 3 partitions per topic). With only two brokers, the cluster must keep serving every partition if either single broker is restarted or fails, while Spark Structured Streaming must resume exactly where it left off after any restart. This project relies on Structured Streaming's checkpointed offsets (persisted to a durable checkpoint volume) as the source of truth for resuming the stream, and on Kafka's per-partition replication across the two brokers for broker-level fault tolerance.

4.2.3 Computing and Partitioning on a Small, Shared Spark Cluster

The Spark Standalone cluster totals only 4 cores across one master and two workers (2 cores / 2 GB each), and those 4 cores are shared between two long-running applications: the Spark Structured Streaming ingestion job (capped at 2 cores) and the Spark Thrift Server that dbt submits its SQL transformations through (also capped at 2 cores). This leaves no spare capacity for ad-hoc Spark SQL at this cluster size, and means both the streaming job's micro-batch partitioning and dbt's batch transformation jobs must fit within their fixed 2-core budget rather than being able to scale out elastically.

Chapter 5

Proposed Solution

5.1 System Architecture

Figure 1 shows the end-to-end architecture: Eventsim generates events into a Kafka cluster, Spark Structured Streaming consumes and lands them as Hudi bronze tables, dbt transforms the bronze layer into a star schema and a wide table on the same Spark cluster, and Superset visualizes the result through the Spark Thrift Server, alongside a Prometheus/Grafana monitoring stack.

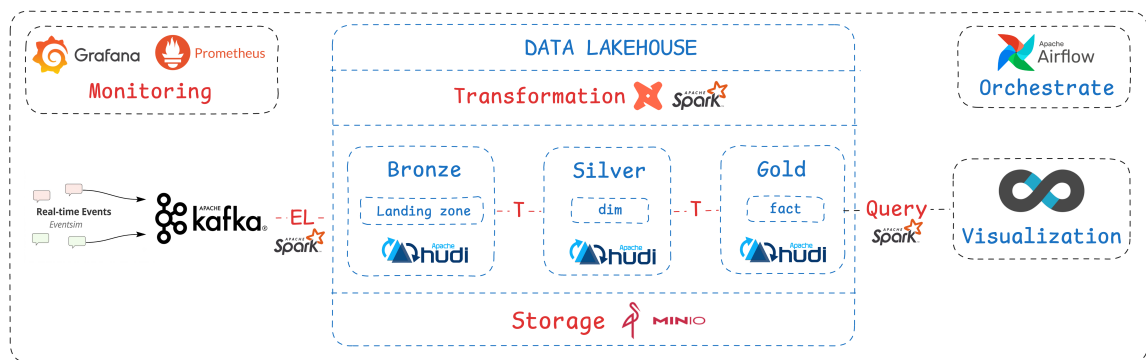


Figure 1: Streamify architecture overview: Eventsim, Kafka, Spark Structured Streaming, Hudi, dbt, Superset, and monitoring.

Table 5.1: Technology stack per pipeline layer.

Layer	Tool	Role
Data simulation	Eventsim	Generates synthetic music-streaming events based on the Million Song Dataset
Message queue	Apache Kafka 4.3 (KRaft)	2-node cluster, replication factor 2, 3 partitions per topic, no ZooKeeper
Stream processing	Spark Structured Streaming 3.5.3	Reads Kafka, decodes/enriches events, writes bronze tables to Hudi
Table format / storage	Apache Hudi	MERGE_ON_READ bronze streaming tables, COPY_ON_WRITE gold/seed tables, on MinIO/S3 via the Hadoop S3A connector
Compute cluster	Spark Standalone (1 master, 2 workers)	Shared by the streaming job and dbt (via Thrift), 4 cores total
Transformation	dbt (dbt-spark, method: thrift)	SQL-based star schema and wide-table modelling, submitted via

5.2 Implementation Workflow

The pipeline runs as the following seven stages:

1. **Data simulation (Eventsim).** Eventsim simulates a real-world user base, generating page-view and song-play events.
2. **Ingestion and messaging (Kafka).** Apache Kafka 4.3 runs in KRaft mode (no ZooKeeper) as a 2-node cluster. Each topic has 3 partitions with replication factor 2 for fault tolerance.
3. **Real-time processing (Spark Structured Streaming).** A long-running Spark job consumes all three topics, fixes latin1-mojibake on text fields, derives time columns, and appends each micro-batch to bronze Hudi tables (`staging.<topic>`), using the `MERGE_ON_READ` table type so streaming appends land as delta log files that are periodically compacted into columnar base files.
4. **Lakehouse storage (Hudi).** Tables are Hudi-managed, backed by MinIO/S3 locally or real S3 in aws mode via the Hadoop S3A connector, with no rebuild needed to switch. There is no separate catalog service: each table is self-describing at its storage path (a `.hoodie/` metadata directory) and is (re-)registered into Spark's catalog idempotently at job startup.
5. **Data transformation (dbt).** dbt [6] connects to the Spark cluster via the Spark Thrift Server (`method: thrift`) and builds a star schema plus a denormalized wide table directly on top of the bronze Hudi tables, materialized as `COPY_ON_WRITE` Hudi tables rebuilt via full overwrite each run.
6. **Business intelligence (Superset).** Apache Superset [4] connects to the same Spark Thrift Server, querying the `wide_streams` model to visualize insights through interactive dashboards.
7. **Monitoring (Prometheus and Grafana).** Prometheus scrapes Kafka consumer-group lag (via `kafka-exporter`), Spark's built-in Prometheus metrics, and per-container stats (via `cAdvisor`); Grafana visualizes all of it across dashboards.

5.2.1 Star Schema and Wide Table

dbt builds two model layers on top of the bronze Hudi tables:

- **Star schema.** One fact table, `fact_streams`, joined to five dimension tables: `dim_users`, `dim_songs`, `dim_artists`, `dim_locations`, and `dim_datetime`. This is the normalized, storage-efficient model (Section 3.1).
- **Wide table (`wide_streams`).** A denormalized table built from the star schema by pre-joining `fact_streams` with all five dimension tables, so that Superset can query a single flat table instead of performing joins at dashboard-query time.



Figure 2: Star schema: `fact_streams` and its five dimension tables.



Figure 3: dbt DAG showing the dependency between bronze sources, the star schema, and wide_streams.

5.2.2 Streaming Job: Kafka to Hudi Bronze

Listing 5.1 shows the core of the Spark Structured Streaming job that reads a Kafka topic, decodes it, and derives the time columns used to partition the bronze Hudi tables.

Listing 5.1: Building a Hudi bronze stream from a Kafka topic (excerpt from `spark_streaming/jobs/stream_events.py`).

```

1 def build_stream(spark, topic, decode_strings):
2     raw = (
3         spark.readStream.format("kafka")
4         .option("kafka.bootstrap.servers", KAFKA_BROKERS)
5         .option("subscribe", topic)
6         .option("startingOffsets", "earliest")
7         .load()
8     )
9
10    parsed = raw.select(
11        F.from_json(F.col("value").cast("string"), SCHEMAS[topic]).alias
12        "partition",
13        "offset",
14    ).select("data.*", "partition", "offset")
15
16    if decode_strings:
17        parsed = (

```

```

18         parsed.withColumn("artist", decode_udf(F.col("artist")))
19         .withColumn("song", decode_udf(F.col("song")))
20     )
21
22     ts_col = F.timestamp_millis(F.col("ts"))
23     return (
24         parsed
25         .withColumn("ts_timestamp", ts_col)
26         .withColumn("year_", F.year(ts_col))
27         .withColumn("month_", F.month(ts_col))
28         .withColumn("day_", F.dayofmonth(ts_col))
29         .withColumn("hour_", F.hour(ts_col))
30         .withColumn("processed_time", F.current_timestamp())
31         .withColumn("_row_key", F.expr("uuid()"))
32     )

```

5.3 Cluster and Environment Setup

The whole stack is deployed with Docker Compose (`docker-compose.yml`), with a `docker-compose.dev.yml` overlay that adds MinIO for local, S3-free development. Spark runs as a Standalone cluster of 1 master and 2 workers, 2 cores / 2 GB each (4 cores total), shared between the streaming ingestion job and the Spark Thrift Server that dbt submits SQL through, each capped at 2 cores.

Chapter 6

Evaluation and Experiments

6.1 Test Environment

The pipeline was exercised on the Docker Compose stack described in Section 5 (Kafka 2-node cluster, Spark Standalone with 1 master and 2 workers, MinIO for local object storage), run on a single local machine.

6.2 Test Cases

Two categories of tests were used to validate the pipeline:

- **Correctness, on a small dataset.** A small batch of Eventsim events is run through the full path – Kafka, Spark Structured Streaming, Hudi bronze, dbt star schema and wide table – and the resulting rows in `fact_streams` / `wide_streams` are checked against the source events for row counts, key fields, and referential integrity against the dimension tables.
- **Runtime and throughput, on a larger volume.** Eventsim is left running for an extended period to build up a larger event volume, and the streaming job’s micro-batch processing time and Kafka consumer lag are observed under sustained load to check that the pipeline keeps up with the producer on the available 4-core cluster.

6.3 Monitoring as Qualitative Evidence

The Grafana dashboards backed by Prometheus provide qualitative evidence of the pipeline’s throughput and health under load, rather than a formal benchmark.

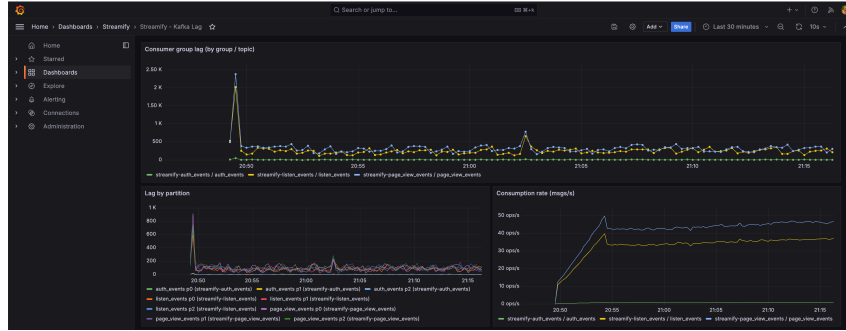


Figure 4: Grafana Kafka Lag dashboard: consumer-group lag and consumption rate per topic.



Figure 5: Grafana Spark dashboard: streaming/batch job metrics, JVM heap, and per-container CPU/memory.

6.4 Scalability

Table 6.1 is a placeholder for measured scalability results (dataset size vs. cluster size vs. runtime). No such measurements exist in this repository at the time of writing (results/ is not present), so no numbers are reported here to avoid fabricating results.

Table 6.1: Scalability results (to be measured).

Dataset	Nodes	Edges	Runtime

Chapter 7

Conclusion

7.1 Results Achieved

This project delivered a working, end-to-end streaming lakehouse pipeline: Eventsim-generated events flow through a 2-node Kafka cluster into a Spark Structured Streaming job that lands MERGE_ON_READ Hudi bronze tables, which dbt transforms (via the Spark Thrift Server) into a COPY_ON_WRITE star schema (fact_streams and five dimension tables) plus a denormalized wide_streams table. Apache Superset dashboards query that wide table through the same Thrift Server, and a Prometheus/Grafana/cAdvisor stack monitors Kafka consumer lag and Spark resource usage throughout. The entire stack is reproducible with a single Docker Compose command.

7.2 Limitations

- **Small, fixed-size cluster.** The Spark Standalone cluster totals only 4 cores, shared between the streaming job and the Thrift Server that dbt uses, leaving no headroom for ad-hoc Spark SQL or additional concurrent workloads.
- **No autoscaling.** Kafka brokers, Spark workers, and other services are fixed at deployment time; the pipeline does not scale resources up or down in response to load.
- **Synthetic dataset.** Eventsim generates realistic-looking but entirely synthetic user

activity over a fixed 10,000-song catalog, so the pipeline has not been validated against real production traffic patterns or data-quality issues.

7.3 Future Directions

- Replace or augment the synthetic Eventsim data with a real or real-scale streaming dataset to validate the pipeline under realistic data-quality and volume conditions.
- Add autoscaling for the Kafka and Spark clusters (e.g. via a container orchestrator) so cluster capacity can grow with load instead of being fixed at deployment time.
- Extend monitoring and alerting beyond the current Grafana dashboards, e.g. automated alerts on consumer lag thresholds, streaming job failures, or dbt run failures.

References

- [1] Apache Software Foundation. *Apache Flink Documentation*. 2024. URL: <https://nightlies.apache.org/flink/flink-docs-stable/> (visited on 09/12/2026).
- [2] Apache Software Foundation. *Apache Hudi Documentation*. 2024. URL: <https://hudi.apache.org/docs/overview> (visited on 09/12/2026).
- [3] Apache Software Foundation. *Apache Kafka Documentation*. 2024. URL: <https://kafka.apache.org/documentation/> (visited on 09/12/2026).
- [4] Apache Software Foundation. *Apache Superset Documentation*. 2024. URL: <https://superset.apache.org/docs/intro> (visited on 09/12/2026).
- [5] Apache Software Foundation. *Structured Streaming Programming Guide*. 2024. URL: <https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html> (visited on 09/12/2026).
- [6] dbt Labs. *dbt Documentation*. 2024. URL: <https://docs.getdbt.com/> (visited on 09/12/2026).
- [7] Snowflake Inc. *Snowflake Documentation*. 2024. URL: <https://docs.snowflake.com/> (visited on 09/12/2026).